free 之后那块内存里有什么：值还在，却已不属于你

嵌入式 Linux 手记　2026-10-16　公众号文章 · 原链接 mp.weixin.qq.com

```c
int *p = malloc(sizeof(int));
*p = 42;
free(p);
printf("%d\n", *p);   /* 先猜：崩溃？0？还是 42？ */
```

`free` 之后又读了一次。在常见的 Linux + glibc 环境下，这段程序多半会安静地打印 42，一个字都没变。此刻的 `p` 已经是悬空指针：它指向的内存所有权已归还分配器，是野指针家族里最常见的一种。

这行 `printf` 是货真价实的未定义行为（C11 §6.2.4 与附录 J.2：对象生命周期结束后使用其指针值，行为未定义），但它的典型表现不是崩，而是演戏。悬空指针（use-after-free）最阴险的地方，恰恰是它不崩。篇 7 结尾那块"没人认领"的内存，现在进去看个究竟。

一、三个反常现象：值还在、不崩、旧值还会穿越

现象一，值还在。`free` 没有任何清零步骤，那 4 个字节原样躺在内存里。你读到 42 不是"应该读到"，而是"还没人动它"的赛跑结果：下一行再来一次 `malloc`，这块地随时可能被新数据覆盖。

现象二，旧值会"穿越"。看一段更诡异的：

```c
char *a = malloc(32);
memcpy(a, "pwd=hunter2", 12);
free(a);

char *b = malloc(32);      /* glibc 上大概率拿回同一块 */
printf("%s\n", b);         /* pwd=hunter2 */
```

刚 `free` 掉的 32 字节，下一次同尺寸申请大概率原址奉还，连里面的旧数据一起奉还。这不是标准承诺，是 glibc 的实现行为（2.26 起的小块走 tcache 缓存，后进先出，机制下一节拆）。安全视角看，这就是为什么口令和密钥用完必须清零：它们不会随 `free` 消失，只会等下一个 `malloc` 来认领。

现象三，同是悬空，换个尺寸就真崩。

```
char *big = malloc(1 << 20);    /* 1MB，超过 mmap 阈值 */
free(big);
*big = 1;                       /* 这一次多半真崩：SIGSEGV */
```

三个现象摆在一起，问题就变成了：`free` 到底做了什么？为什么它既能放任旧值存在，又能当场翻脸？

二、free 到底做了什么：三件事，三不碰

glibc 的 `malloc` 并非系统调用，它是个库函数：向内核批发内存，再向应用零售。`free` 也不是"把内存还给系统"，它做的只是三件簿记：

1. 翻账本：`malloc` 返回的指针前面垫着 chunk 头部，记录本块大小与邻居占用标志位。`free` 靠它知道"这块多大、前一个邻居空闲吗"，这也是 `free` 不用传 size 的原因。
2. 挂链子：把这块空闲块插进空闲链。glibc 2.26 起小块先走 tcache（每线程缓存），后进先出，分配时 O(1) 直接摘走；每个 tcache 桶只存 7 块，装不下的去 fastbin 等后备队列排队。
3. 看邻居：小块走 tcache/fastbin 快速通道（不合并），落到常规空闲链后相邻块空闲就合并成大块，抑制碎片；glibc 对最常见的重复 `free` 也有检测，能拦住的当场 abort，拦不住的照样未定义。

三不碰：不清零（内容原样保留）、不归还 OS（小块进空闲链等复用，只有堆顶累计空闲超过阈值才可能收缩归还；mmap 大块是例外，下一节见）、不通知指针（`p` 还是原来那个值，只是它指向的地已经易主）。

为什么不清零？清零是真金白银的写流量，谁需要干净内存谁自己来，`calloc` 自带清零。为什么急着复用？后进先出意味着刚 `free` 的块大概率还在 CPU 缓存里，是热的；摘链一步到位，还省掉在空闲链里翻找合适块的功夫。分配快、命中高，tcache 的设计动机就在这，不是巧合。



图：free 只改头部标志、把块挂进 tcache，数据区原样不动

三、悬空的下场：分配渠道决定崩不崩

三个现象，一把钥匙：那块内存当时躺在哪个渠道。

小块走 `brk` 批发的堆：`free` 只改账本挂链子，那几页虚拟地址依然映射、物理页依然存在，读写它 MMU 一路绿灯。所以不崩，值还在。大块（glibc 默认 128KB 以上）走 `mmap` 独立映射：`free` 即 `munmap`，映射当场解除，访问它就是访问一片没有映射的地址，MMU 直接拒绝，段错误，当场崩。

同一个悬空指针，两种下场。"有时崩有时不崩"不是玄学，是渠道。

但"不崩"只是暂时的。tcache 急着复用，下一次 `malloc` 就可能把这块地翻新：你读到的不再是 42，而是别人的 int；你写进去的 42 也悄悄污染了别人的数据。崩溃尚有栈可查，这种静默的数据错乱没有崩溃现场，是 use-after-free 难查的根源。浏览器内核的 CVE 榜单上 use-after-free 长期霸榜，安全研究者更怕的正是崩与不崩之间的那种。

UB 的边界再钉清楚一步。C 标准直到 C23 才专门写清这条线：无效指针值可以无条件复制，但复制之外的任何使用（首先是解引用）都是 UB；C23 之前标准对此着墨不明。无论如何演进，"解引用悬空指针"从 C89 到今天始终是板上钉钉的 UB。

还要多想一层：UB 不只是"运行时可能出事"。开 `-O2` 后编译器有权假定 UB 不发生，直接把"读了已释放内存"的代码折叠掉，你在源码里看到的逻辑和可执行文件里跑的，可能根本不是同一套。所以"值还在"连编译器这关都过不了，它只是恰好的赛跑结果。



图：brk 堆上悬空不崩，mmap 大块 free 即解除映射，访问必崩

四、防御：让误用崩在第一现场

第一招，释放后立即置空。

```
free(p);
p = NULL;
```

之后再误用，`p` 是空指针，Linux 这类平台上解引用几乎必崩，栈帧直接指向肇事行。悬空指针读旧值时那些"看起来正常"的日子，才是最难查的。让 bug 崩得早，是工程上划算的交易。（空指针为什么几乎必崩，内存系列后续专篇拆解。）

第二招，敏感数据先清零再释放。`memset` 有被编译器优化掉的风险：死存储消除会认定"没人再读这块内存"，把清零整个删掉。口令、密钥这类数据用 `explicit_bzero`（glibc 2.25+），语义上保证清零不被删除。

第三招，编译期和测试期各上一道闸。GCC 13 起提供 `-Wuse-after-free`，对 `free` 之后直接使用指针的典型模式发警告；怀疑有漏网的，用 AddressSanitizer（`-fsanitize=address`）跑一遍测试。它的思路和 glibc 反着来：每块内存前后垫红区，`free` 的块扔进隔离区延迟复用。生产环境里"急着复用让 bug 隐身"，测试环境里"拒绝复用让 bug 现形"，悬空读写当场报错，附完整分配与释放的调用栈。

第四招，把所有权写进契约。谁分配谁释放，接口注释写清；`realloc` 可能搬移，搬移后旧指针即刻悬空，只允许使用返回值。

写在最后

1. free 做三件事：读 chunk 头、挂空闲链、合并邻居；三不碰：不清零、不归还 OS、不通知指针。
2. 悬空崩不崩取决于分配渠道：brk 堆上页仍映射，多半安静；mmap 大块 free 即解除映射，访问必崩。
3. free 后立即 p = NULL，让误用崩在第一现场；敏感数据用 explicit_bzero，别赌 memset 不被优化。
4. "读到旧值"是赛跑结果不是承诺：下一次 malloc 随时翻新这块内存，静默数据错乱比崩溃更难查。
5. GCC 13 的 -Wuse-after-free 加测试期的 ASan（红区 + 隔离区），把隐身的悬空访问逼到台前。

堆上的悬空至少要等一次 `free`；栈上的悬空随手就能造出来：`return` 一个局部数组的地址，函数一返回，它就悬空了。返回指针有四条死路，三条踩了就炸，一条最隐蔽。下一篇《函数返回指针的四条死路》。

合集持续更新，往期与后续篇目都在目录里：C 语言底层实战 · 合集目录

后台回复「交流」，我很乐意分享我的学习计划和一线工作经验，直接来聊。